

Digital System Design and Implementation

Final Exam(112/05/30)

Spring 2023

Total points: 100 points

- Please **download** the template file DSD.v, which contains all the required I/O in each question and **rename** it as DSDxxxxxxxx.v or DSDxxxxxxxx.sv, where x is your student ID, for example, DSD105501001.v.
- Use the template to ensure your module and input as well as output ports having **the same module/port names** as described in the following.
- **Open book, but discussion is not allowed.**

In this final exam, we want to have some maximum search among elements defined by certain 2-dimensiona pattern. Please write each module first.

1. Please write a rotator that generate the output pattern as defined in the following figure. The inputs and outputs are all 8 bits. (10%)

module Rotaor(Out3, Out2, Out1, In3, In2, In1, rMode);

endmodule

Out1	Out2	Out3	
In1	In2	In3	rMode=0
In2	In3	In1	rMode=1
In3	In1	In2	rMode=2

2. Please write a synchronous read/write register file that has three read ports (QA, QB, QC) and one write port (D) to save an image with size up to 1023. Each word has 8 bits wide. The write enable signal “**WE**” is active high and the meaning of the address is defined in the Verilog template. (10%)

module RegisterFile(QA, QB, QC, D, Clk, RAA, RBA, RCA, WE, WA);

endmodule

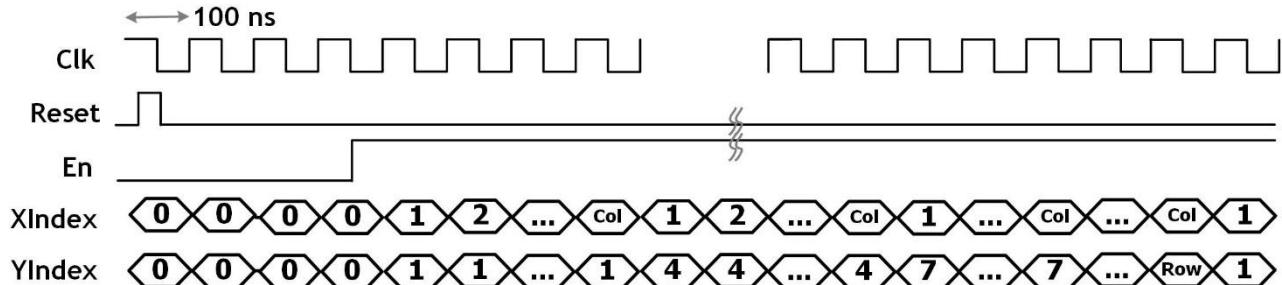
3. Please write a two-dimensional index counter generating outputs counting from (1,1) to (COL, ROW) when ‘En’ is True, where ROW and COL are defined as shown in the following. The timing diagram must fulfill the following figure. Note that asynchronous reset is used. Because we will process 3 rows at a time subsequently, please pay attention to the spacing of YIndex in the figure. (12%)

```

module Count2D(XIndex, YIndex, Clk, Reset, En);
    parameter COL=50; // COL is smaller than 128, 0<=XIndex<=COL
    parameter ROW=25; // ROW is smaller than 64, 0<=YIndex<=ROW
    input Clk, Reset, En;

endmodule

```



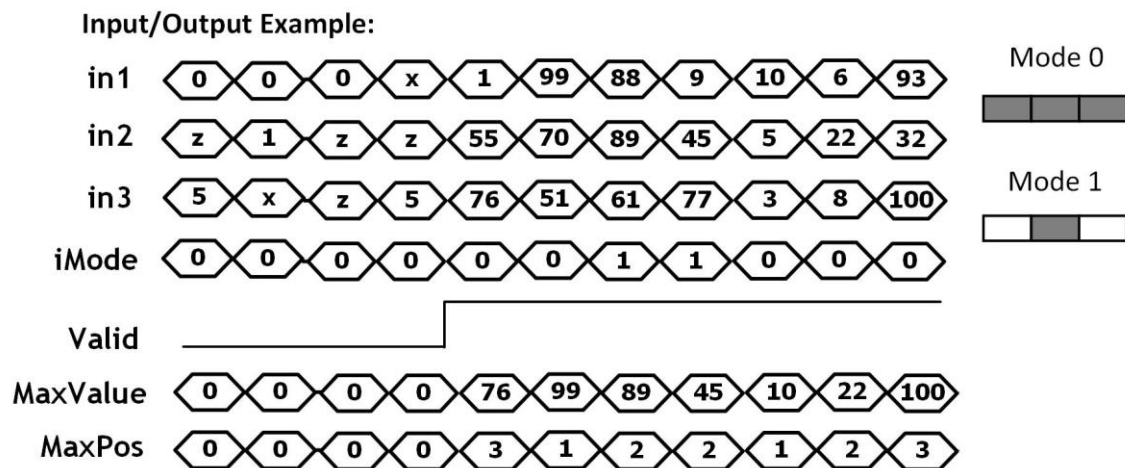
4. Please write a module called “**RowComparator**” that has two modes to decide the maximum and its position in each mode among the three input as shown in the figure. Note that if there is unknown or Hi-Z in the three inputs, the “Valid” signal will be low. (15%)

```

module RowComparator(MaxValue, MaxPos, Valid, In1, In2, In3, iMode);
    input [7:0] In1, In2, In3; //Only one maximum exist
    input iMode; //iMode=0, compare three inputs; iMode=1, send out center input

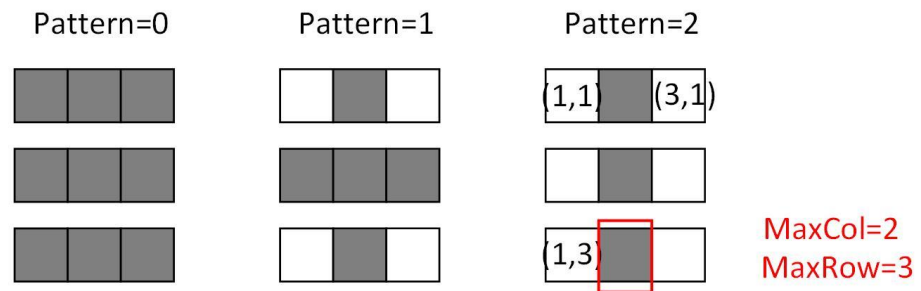
endmodule

```



5. Now, we want to process three rows simultaneously that have 9 elements in total to decide the maximum among the elements in the following three patterns defined by the shadowed area. Note that you must use “**RowComparator**” in Q4. The output signals “MaxRow”, “MaxCol” indicates the position of the maximum and $1 \leq \text{MaxRow}, \text{MaxCol} \leq 3$, where input signal In2_3 means

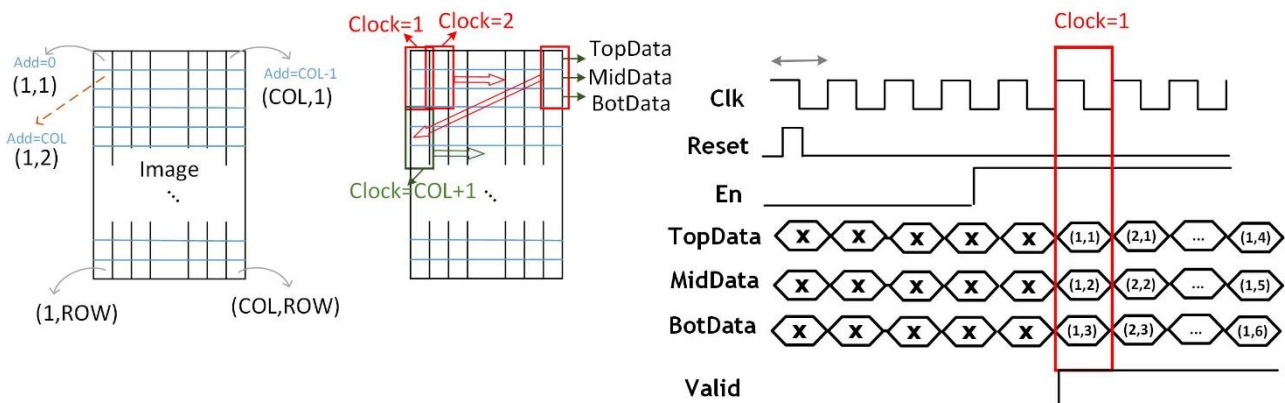
the 2nd column and the third row. If it is the maximum, the outputs of MaxRow and MaxCol are shown in the figure. (20%)



```
module PatternComparator3x3(MaxValue, MaxRow, MaxCol, Valid, In1_1, In2_1, In3_1, In1_2, In2_2, In3_2, In1_3, In2_3, In3_3, Pattern);
input [7:0] In1_1, In2_1, In3_1;
input [7:0] In1_2, In2_2, In3_2;
input [7:0] In1_3, In2_3, In3_3;
input [1:0] Pattern;

endmodule
```

6. Now, we will rely on the inputs generated from the two-dimensional counter (**Count2D**) to fetch three consecutive elements from the register file (**RegisterFile**) according to the sequence shown in the following figure. Note that we will put data into the register file through test bench. You can assume that it contains effective data now. You must use **RegisterFile**, to complete the design of module **DataFetch**. Note that for clarity, the 2D indexes of TopData, MidData, and BotData are drawn in the Figure. Actually, the associated values will be sent out. (13%)



```
module DataFetch(Valid, TopData, MidData, BotData, Xindex, Yindex, En, Clk, Reset);
input Clk, Reset, En;

RegisterFile U6_1();

endmodule
```

7. Now, please complete the design for maximum search among an 2D image for a given pattern. (a) Three shift-registers are used each having length of 3 to store the 3×3 data of 2D image fetched

from register file. (b) A maximum and its associated X Index (*MaxXPos*) and Y Index (*MaxYPos*) in the 2D image will be generated according to the pattern defined in the input. Note that the module (*DataFetch*) must be used. Note that the “*MaxValid*” will be asserted only when the complete data in three consecutive rows are all in the shift register for the maximum search. (20%)

module MaxSearch2D(MaxValue, MaxXPos, MaxYPos, MaxValid, Clk, Reset, En, Pattern);

end

